Topics:

Forgot Password Page Route, Token Generation using URLSafeTimedSerializer, Sending the Reset Password Link via Email, Reset Password Link Handling, Flash Messages for Feedback.

working flow of your Forgot Password + Reset Password with Token application in simple step-by-step:

✅ 1. User Opens Login Page

- **The user visits:**
- **http://localhost:5000/**
- **Flask returns: login.html**

If the user enters:

- **Correct email & password → dashboard opens.**
- **Wrong details → flash message: *"Invalid Login!"***

✅ 2. User clicks "Forgot Password?"

- **@app.route('/forgot_password') renders forgot_password.html**
- **This page asks the user to enter email.**

✅ 3. User Submits Email to Reset Password

Form submits to:

@app.route('/send_reset_link', methods=['POST'])

What happens here:

1. **Flask checks if email exists in users dictionary.**
2. **If not found → flash: *"Email not registered!"***
3. **If found → application creates a token:**

token = s.dumps(email, salt='password-reset-salt')

This token is encrypted email.

4. **It creates a reset link:**

http://localhost:5000/reset_password/<token>

5. **Flask sends email using this:**

msg = Message("Password Reset Request", sender="YourEmail", recipients=[email])

mail.send(msg)

6. **Flash message: *"Reset link sent to your email!"***

✅ 4. User Opens the Reset Link from Email

When user clicks the link → Flask goes to:

@app.route('/reset_password/<token>')

Here:

email = s.loads(token, salt='password-reset-salt', max_age=300)

- **It decrypts token to get original email.**
- **max_age=300 → link is valid for 5 minutes.**
- **If link expired → Message: *"Link expired! Try again."***

✅ 5. User Sets New Password

If page loaded successfully → user sees reset_password.html

After entering new password → form submits again:

users[email]["password"] = new_password

So their password is updated in dummy database.

Flash message:

"Password reset successful! Please login."

User is redirected to login page.

✅ 6. User Logs In With New Password

Login works again normally

✅ **7. Logout**

When the user clicks Logout:

session.pop('email', None)

User session is removed → redirected to login page.

Flow Diagram (Super Simple)

Login Page → Forgot Password → Enter Email →

Check Email Exists → Generate Token → Send Email →

|

↓

User Clicks Email Link → Reset Form → Update Password → Login Again

---

Folder Structure:

your_project_folder/

|

├── app.py # Your main Flask application file

|

├── templates/ # All HTML files go here

| ├── login.html

```
│   ├── dashboard.html
│   ├── forgot_password.html
│   └── reset_password.html
```

app.py:

```
from flask import Flask, render_template, request, redirect, flash, session
from flask_mail import Mail, Message
from itsdangerous import URLSafeTimedSerializer, SignatureExpired

# Create Flask application
app = Flask(__name__)
app.secret_key = "mysecretkey"   # Secret key for session and token encryption

# ---------------- EMAIL CONFIGURATION ----------------
app.config['MAIL_SERVER'] = 'smtp.gmail.com'   # Gmail SMTP server
app.config['MAIL_PORT'] = 587                  # Mail server port
app.config['MAIL_USE_TLS'] = True              # Use TLS encryption
app.config['MAIL_USERNAME'] = "janicode249@gmail.com"  # Sender email
app.config['MAIL_PASSWORD'] = "iucf kdgo dgcs digr"     # App Password from Gmail

mail = Mail(app)  # Initialize Flask-Mail with app

# Token Serializer (used for generating secure reset password links)
s = URLSafeTimedSerializer(app.secret_key)

# Dummy Database (Dictionary for example only — no real DB)
users = {"janicode249@gmail.com": {"password": "abc123"}}
```

```
# ---------------- ROUTES ----------------

# Login Page
@app.route('/')
def home():
    return render_template("login.html")



# Login Handling
@app.route('/login', methods=['POST'])
def login():
    email = request.form['email']       # Get user email from form
    password = request.form['password']  # Get password from form

    # Check if user exists and password matches
    if email in users and users[email]["password"] == password:
        session['email'] = email  # Store email in session (User stays logged in)
        return redirect("/dashboard")  # Redirect to dashboard
    else:
        flash("Invalid Login!")  # Show error message
        return redirect("/")     # Redirect back to login page



# Dashboard Page (Accessible only if logged in)
@app.route('/dashboard')
def dashboard():
```

```
    if 'email' in session:   # If user is logged in
        return render_template("dashboard.html", user=session['email'])
    return redirect('/')     # If not logged in → go to login


# Forgot Password Page
@app.route('/forgot_password')
def forgot_password():
    return render_template("forgot_password.html")


# Sending Reset Password Email Link
@app.route('/send_reset_link', methods=['POST'])
def send_reset_link():
    email = request.form['email']  # Get entered email

    # If email is not present in dummy database
    if email not in users:
        flash("Email not registered!")
        return redirect('/forgot_password')

    # Create secure token containing user's email
    token = s.dumps(email, salt='password-reset-salt')

    # Generate reset link using token
    link = f"http://localhost:5000/reset_password/{token}"

    # Create email message
```

```
    msg = Message("Password Reset Request",
                  sender="janicode249@gmail.com",
                  recipients=[email])
    msg.body = f"Click the link to reset your password: {link}"
    mail.send(msg)  # Send email

    flash("Reset link sent to your email!")
    return redirect('/')


# Reset Password Page (User clicks the link from email)
@app.route('/reset_password/<token>', methods=['GET', 'POST'])
def reset_password(token):
    try:
        # Decode token to get user's email
        email = s.loads(token, salt='password-reset-salt', max_age=300)
        # max_age=300 → link valid for 5 minutes
    except SignatureExpired:
        return "Link expired! Try again."

    # After form submission → Save new password
    if request.method == 'POST':
        new_password = request.form['password']
        users[email]["password"] = new_password  # Update password in dummy DB
        flash("Password reset successful! Please login.")
        return redirect('/')

    # Show Reset Password Form
```

```
    return render_template("reset_password.html")



# Logout
@app.route('/logout')
def logout():
    session.pop('email', None)  # Remove user session
    return redirect('/')       # Go back to login page



# Run server
app.run(debug=True)
```

---

login.html

```
<!DOCTYPE html>
<html>

<body>



    <h2>Login</h2>
    <!-- Flash Messages Here -->
{% with messages = get_flashed_messages() %}
  {% if messages %}
    {% for msg in messages %}
      <p style="color:red;">{{ msg }}</p>
    {% endfor %}
  {% endif %}
```

```
{% endwith %}
   <form action="/login" method="POST">
      Email: <input type="email" name="email"><br><br>
      Password: <input type="password" name="password"><br><br>
      <button type="submit">Login</button>
   </form>
   <br>
   <a href="/forgot_password">Forgot Password?</a>
</body>

</html>
```

Dashboard.html

```
<!DOCTYPE html>
<html>
<body>
<h2>Welcome {{ user }}</h2>
<a href="/logout">Logout</a>
</body>
</html>
```

---

forgot_password.html

```
<!DOCTYPE html>
<html>
<body>

<h2>Forgot Password</h2>
<!-- Flash Messages Here -->
{% with messages = get_flashed_messages() %}
```

```
  {% if messages %}
    {% for msg in messages %}
      <p style="color:red;">{{ msg }}</p>
    {% endfor %}
  {% endif %}
{% endwith %}
<form action="/send_reset_link" method="POST">
  Enter Registered Email: <input type="email" name="email"><br><br>
  <button type="submit">Send Reset Link</button>
</form>
</body>
</html>
```

---

reset_password.html

```
<!DOCTYPE html>
<html>
<body>

<h2>Reset Your Password</h2>
<!-- Flash Messages Here -->
{% with messages = get_flashed_messages() %}
  {% if messages %}
    {% for msg in messages %}
      <p style="color:red;">{{ msg }}</p>
    {% endfor %}
  {% endif %}
{% endwith %}
```

```
<form method="POST">
  New Password: <input type="password" name="password"><br><br>
  <button type="submit">Reset</button>
</form>
</body>
</html>
```

Flask Information:

Even though you are flashing **one message**, Flask is designed to handle **multiple flash messages**, so get_flashed_messages() always returns a **list**.

That is why we use a **for loop**.

Let's understand step-by-step.

1️⃣ What flash() does

In **Flask**, flash() stores messages temporarily in the **session**.

Example in app.py:

```
from flask import flash

flash("login successfully done")
```

This message is stored in a **list inside the session**.

Example internally:

```
["login successfully done"]
```

But sometimes you may store **multiple messages**.

Example:

```
flash("Login successfully done")
flash("Welcome Jani Basha")
flash("Profile updated")
```

Now the session contains:

```
[
 "Login successfully done",
 "Welcome Jani Basha",
 "Profile updated"
]
```

2️⃣ What get_flashed_messages() returns

When you call:

```
messages = get_flashed_messages()
```

It always returns a **list**.

Example:

```
["Login successfully done"]
```

or

```
["Login successfully done", "Welcome Jani Basha"]
```

3️⃣ Why we use a for loop

Since messages is a **list**, we must iterate through it.

```
{% for msg in messages %}
   <p style="color:red;">{{ msg }}</p>
{% endfor %}
```

Example output if there are 3 messages:

```
Login successfully done
Welcome Jani Basha
Profile updated
```

4️⃣ What happens if we don't use the loop?

If you write:

```
{{ messages }}
```

Output will be:

['Login successfully done']

Which is **not user friendly**.

5️⃣ Simple Visualization

```
flash()  ---> stores messages in session list
          ↓
get_flashed_messages()
          ↓
["msg1", "msg2", "msg3"]
          ↓
for loop
          ↓
Display each message
```

6️⃣ Final Template (Best Practice)

```
{% with messages = get_flashed_messages() %}
  {% if messages %}
    {% for msg in messages %}
      <p style="color:red;">{{ msg }}</p>
    {% endfor %}
  {% endif %}
{% endwith %}
```

✅ Key Point

Even if you flash **one message**, Flask returns it inside a **list**, so we use a **loop** to handle **one or many messages safely**.